

Appendix L — API Endpoints Reference

`API\_ENDPOINTS.txt` was a real, 143-line file the project maintained at its working root — first written 2026-06-20, last edited 2026-06-27 alongside a feedback-endpoint rework — but it lived outside this book repository's snapshot and is not present on disk here. What follows is reconstructed directly from the two servers' current source (`app/api.py` and `app\_workflow/api.py`), cross-checked against the original file's structure as preserved in the chat transcript that authored it, and it deliberately does not pretend the two servers have stayed in sync since — one real divergence between them is called out explicitly in §L.5, because a reader integrating against both would hit it immediately.

L.1 The dual-port setup

Pipeline	Port	Started from
LangChain (`app/api.py`)	8000	`cd app && python api.py`
LangGraph (`app_workflow/api.py`)	8001	`cd app_workflow && python api.py`
Marker microservice (`marker_service/server.py`)	8002	Docker; optional GPU sidecar for `/ingest`, Chapter 41B

Both ports are hardcoded in each file's `uvicorn.run(...)` call, not read from `config.py` — there is no `PORT` constant in either config module. Every request to either server follows the same convention: `Content-Type: application/json`, raw JSON body, changing only the port between the two.

L.2 LangChain pipeline — app/api.py (port 8000)

Startup builds the embedding manager, vector store, retriever, the three `app/`-side LLM roles (Appendix N), tool schemas, `FeedbackStore`, and `SelfLearner`, and refuses to start (`RuntimeError`) if the vector store is empty.

POST /query

```
class QueryRequest(BaseModel):
    query: str
```

Rejects a blank query with `400`. Generates a `request\_id`, loads prior failed variants and thumbdowns for this normalized question (Appendix D), and runs the agent off-thread so the event loop is never blocked by a synchronous agent call:

```
result, newly_failed = await asyncio.to_thread(
    run_agent, req.query, llm, merge_llm, judge_llm, tool_schemas,
    callables,
    check_answer_quality, retriever, agent_state, blocked,
    prior_thumbdowns, request_id,
)
```

```
{
  "answer": "string",
  "sources": [{"query": "string", "preview": "string"}],
  "iterations": 2,
  "quality": "OK | INSUFFICIENT",
  "document_chunks": [{"content": "string", "source": "string"}],
  "learned_qa_chunks": [{"content": "string", "source": "string"}],
  "variants": [{"query": "...", "document_chunks": [...],
"learned_qa_chunks": [...]}],
  "total_prompt_tokens": 1234,
  "total_completion_tokens": 456,
  "request_id": "uuid"
}
```

A rate-limit abort (Appendix M) surfaces here as `429`, with `Retry-After` set to the required backoff in seconds and `detail` stating the same number in prose.

POST /feedback/bad

```
class FeedbackRequest(BaseModel):
    request_id: str
    feedback: str = ""
```

`404` with `detail="Interaction not found: request\_id=..."` if no matching record exists. `persisted` is `true` only when `feedback` is at least 10 characters (the project's `MIN\_FEEDBACK\_LEN`).

```
{  "status":    "flagged",    "request_id":    "uuid-from-query-response",
  "persisted": true }
```

GET /stats, POST /learn, POST /quit

```
GET    /stats    →    {  "total_interactions":  42,  "successful":  38,
  "learned_qa_pairs": 10,
                        "auto_distillation_enabled": false, "learn_every_n": null
  }
POST /learn → { "added_qa_pairs": 3 }
POST /quit  → { "status": "shutting down" }    (sends SIGTERM to its own
process)
```

L.3 LangGraph pipeline — app_workflow/api.py (port 8001)

`app\_workflow/routes.py` is not an HTTP router — despite the name, it holds the graph's conditional-edge routing functions (`route\_after\_combine`, `route\_after\_quality\_check`). All six HTTP endpoints for this pipeline live in `api.py` itself.

POST /query — per-request feature-flag overrides

This is the one real capability `app`'s `/query` has no equivalent of: a caller can override any of the pipeline's twenty `ENABLE\_\*` feature flags for a single request, without touching `config.py`. The complete model, every field:

```
class WorkflowSwitches(BaseModel):
    """Per-request overrides for the ENABLE_* toggles in config.py.

    Any field left as None keeps the config.py default for that switch.
    """

    ENABLE_SUB_QUERY_GENERATION: Optional[bool] = None
    ENABLE_QUERY_VARIANTS_OUTPUT_FIX: Optional[bool] = None
    ENABLE_RETRIEVAL_DEDUP_MERGE: Optional[bool] = None
    ENABLE_RETRIEVAL_DEDUP_MERGE_OUTPUT_FIX: Optional[bool] = None
    ENABLE_RETRIEVAL_DEDUP_MERGE_VALIDATION: Optional[bool] = None
    ENABLE_RETRIEVAL_DEDUP_MERGE_VALIDATION_OUTPUT_FIX: Optional[bool] =
None
    ENABLE_RETRIEVAL_VALIDATION: Optional[bool] = None
    ENABLE_RETRIEVAL_VALIDATION_OUTPUT_FIX: Optional[bool] = None
    ENABLE_NAC_COMPRESSION: Optional[bool] = None
    ENABLE_DC_COMPRESSION: Optional[bool] = None
    ENABLE_LBC_COMPRESSION: Optional[bool] = None
    ENABLE_COMPRESSION_VALIDATION: Optional[bool] = None
    ENABLE_COMPRESSION_OUTPUT_FIX: Optional[bool] = None
    ENABLE_ANSWER_DRAFT_CREATION: Optional[bool] = None
    ENABLE_ANSWER_QUALITY_CHECK: Optional[bool] = None
    ENABLE_ANSWER_QUALITY_OUTPUT_FIX: Optional[bool] = None
    ENABLE_AUTO_DISTILLATION: Optional[bool] = None
    ENABLE_QA_PAIR_GENERATION: Optional[bool] = None
    ENABLE_QA_PAIR_OUTPUT_FIX: Optional[bool] = None
    ENABLE_GLOBAL_LLM_OUTPUT_FIX: Optional[bool] = None

class QueryRequest(BaseModel):
    query: str
    switches: Optional[WorkflowSwitches] = None
```

Reserved command words (`bad`, `stats`, `learn`, `exit`, `quit`) are rejected with `400` rather than being treated as a query, and the graph itself is invoked off-thread the same way `app/`'s agent is:

```
final_state = await asyncio.to_thread(
    rag_app.invoke, state, {"callbacks": [langfuse_handler] if
    langfuse_handler else []},
)
```

```
{
  "answer": "string",
  "quality": "OK | <verdict string>",
  "sources": [{"source": "string"}]
}
```

Common pitfall — This response is deliberately smaller than app/'s

`request\_id` is generated and threaded through the entire graph run via `set\_request\_id()`, but this endpoint never echoes it back to the caller — and there is no `document\_chunks`, `learned\_qa\_chunks`, `variants`, or token-count field either, all of which `app/`'s `/query` does return. A client written against one pipeline's response shape and pointed at the other's port will silently lose fields, not error — check which pipeline you are actually talking to before assuming a response key exists.

L.4 The remaining LangGraph endpoints

`GET /stats` and `POST /learn` match `app/`'s shapes; `POST /learn` additionally threads a Langfuse callback config through that `app/`'s does not:

```
learn_config = {"callbacks": [langfuse_handler]} if langfuse_handler else
None
added = await asyncio.to_thread(self_learner.run_distillation,
    config=learn_config)
```

POST /ingest is the one endpoint with no counterpart on the LangChain side at all (added 2026-07-24, Chapter 41B):

```
class IngestRequest(BaseModel):
    ENABLE_MARKER_LOADER: Optional[bool] = None
    ENABLE_CUSTOM_SPLITTER: Optional[bool] = None
```

```
{
  "files_discovered": 12,
  "documents_loaded": 12,
  "chunks_created": 340,
  "documents_in_store": 340,
  "marker_loader": false,
  "custom_splitter": false
}
```

When `ENABLE\_MARKER\_LOADER` resolves true, the separate Marker microservice (port 8002, Table L.1) must already be running — `/ingest` calls out to it rather than loading `marker-pdf` in-process (ADR-073, Chapter 41B).

L.5 A live asymmetry: POST /feedback/bad

`app`'s `/feedback/bad` was reworked on 2026-06-27 (ADR-047) to require a `request\_id` and look the interaction up directly in MongoDB, removing the earlier constraint that it had to be called immediately after `/query` in the same process. `app\_workflow`'s `/feedback/bad` was never migrated to that scheme — it still relies on a process-global “last result” sidecar:

```
# app_workflow/api.py
class FeedbackRequest(BaseModel):
    feedback: str = ""

variants = list(services.last_variants_with_chunks)
ok = feedback_store.mark_last_bad(feedback=req.feedback, variants=variants)
if not ok:
    raise HTTPException(status_code=404, detail="No previous answer to flag.")
```

Common pitfall — Same endpoint name, two different contracts

Calling `POST /feedback/bad` against port 8001 without having just called `/query` on the same running process returns `404 "No previous answer to flag."` — the `request\_id`-based lookup that makes port 8000's version tolerant of any ordering or delay simply does not exist on this pipeline. `Decisions.md`'s ADR-047 entry scopes the fix explicitly to `app/api.py`; `mark\_last\_bad()` was deliberately kept for `app\_workflow`'s internal CLI callers (its own `cmd\_bad` node uses the identical mechanism), and the HTTP route simply inherited that same constraint rather than getting its own `request\_id` variant. Treat the two pipelines' `/feedback/bad` as genuinely different contracts, not two ports of the same endpoint.

L.6 Postman recipes

No exported Postman collection exists anywhere in this project — the original `API_ENDPOINTS.txt`'s own “Postman setup” section was two lines, not a `.postman_collection.json` file, and that is the honest scope of what this section can offer: the same two-line convention, applied to the real request shapes documented above.

- New request → Method and URL from any endpoint above (swap port 8000/8001 for the two pipelines).
- Headers tab → one entry: Content-Type: application/json.
- Body tab → raw, type JSON, paste the matching request schema from §L.2–L.4 (omit the body entirely for GET `/stats` and POST `/learn/quit`).
- Save each request into two folders, one per pipeline, so the identical `/query` and `/feedback/bad` names don't collide in Postman's own history — a small precaution that matters more than usual given §L.5's real divergence between the two.